

LAB REPORT 5.2

Custom Topologies with the Mininet Python API

Student Name	Hadis Rodpradit
Student ID	6630613039
Program	Digital Engineering International Program, College of Computing, PSU Phuket
Course	977-337 Network Programming and Simulation
Week	5
Environment	Ubuntu Linux VM, Mininet Python API, Open vSwitch, default controller <code>c0</code>
Source file	<code>lab05_custom_topo/custom_topo.py</code> in the netprog-2026 repository
Date	29 September 2026

1. Objective

In this lab I described topologies in Python instead of using the `mn --topo` generators. I subclassed `Topo` and used `addHost()`, `addSwitch()` and `addLink()` to build a minimal topology and an asymmetric two-switch topology, then used `TCLink` to turn the s1-s2 backbone into a constrained WAN link (5 Mbit/s, 50 ms delay, 10% loss) and verified its effect with `iperf`, `ping` and `pingall`.

2. Lab Tasks and Results

Task 1 — Minimal Custom Topology (Parts B-C)

`MinimalTopo.build()` creates hosts `h1` and `h2`, one switch `s1`, and links both hosts to `s1`. The script was run with `sudo python3 custom_topo.py`.

```
class MinimalTopo(Topo):
    def build(self):
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        s1 = self.addSwitch('s1')
        self.addLink(h1, s1)
        self.addLink(h2, s1)
```

```

diz@diz:~/netprog-2026/lab05_custom_topo$ nano custom_topo.py
diz@diz:~/netprog-2026/lab05_custom_topo$
diz@diz:~/netprog-2026/lab05_custom_topo$ sudo python3 custom_topo.py
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
*** Starting CLI:
mininet>
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0
c0
mininet>
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done

```

Figure 1: MinimalTopo: net.pingAll() reports 0% dropped (2/2), and net shows h1 and h2 on s1

`net.pingAll()` printed **0% dropped (2/2 received)** before the `mininet>` prompt appeared, and `net` confirms the shape written in code: `h1-eth0:s1-eth1` and `h2-eth0:s1-eth2`.

Task 2 — Asymmetric Custom Topology (Part D)

```
diz@ediz:~/netprog-2026/lab05_custom_topo$ nano custom_topo.py
diz@ediz:~/netprog-2026/lab05_custom_topo$
diz@ediz:~/netprog-2026/lab05_custom_topo$
diz@ediz:~/netprog-2026/lab05_custom_topo$ sudo python3 custom_topo.py
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s2) (s1, s2)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
*** Starting CLI:
mininet>
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s2-eth1
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:s2-eth2
s2 lo: s2-eth1:h4-eth0 s2-eth2:s1-eth4
c0
mininet>
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 5 links
.....
*** Stopping 2 switches
s1 s2
*** Stopping 4 hosts
h1 h2 h3 h4
*** Done
```

Figure 2: CustomTopo: net output shows 3 hosts on s1 and 1 host on s2; pingall 0% dropped

The `net` output shows the asymmetric shape: **s1** has four ports — three to hosts (**s1-eth1:h1**, **s1-eth2:h2**, **s1-eth3:h3**) and one to the backbone (**s1-eth4:s2-eth2**) — while **s2** has only **h4** plus the backbone port. All four hosts reach each other (**0% dropped, 12/12**). None of the built-in generators can produce this, because single, linear and tree always attach the same number of hosts to each switch.

Part E — Final Script with TCLink

To reach the targets (≈ 5 Mbit/s, ≈ 100 ms extra RTT, occasional drops) I chose `bw=5`, `delay='50ms'`, `loss=10` on the s1-s2 link only, and passed `link=TCLink` to `Mininet()`. `bw` is in Mbit/s. TCLink applies the delay to the interface at *each* end of the link, so a packet is delayed 50 ms going out and its reply is delayed 50 ms coming back, giving $50 + 50 = 100$ ms of extra RTT.

```

from mininet.net import Mininet
from mininet.topo import Topo
from mininet.cli import CLI
from mininet.log import setLogLevel
from mininet.link import TCLink

class CustomTopo(Topo):
    def build(self):
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        h3 = self.addHost('h3')
        h4 = self.addHost('h4')

        s1 = self.addSwitch('s1')
        s2 = self.addSwitch('s2')

        self.addLink(h1, s1)
        self.addLink(h2, s1)
        self.addLink(h3, s1)
        self.addLink(h4, s2)
        # TODO 7: constrained WAN backbone link
        self.addLink(s1, s2, bw=5, delay='50ms', loss=10)

def main():
    setLogLevel('info')
    # TODO 8: link=TCLink so bw/delay/loss actually apply
    net = Mininet(topo=CustomTopo(), link=TCLink)
    net.start()
    net.pingAll()
    CLI(net)
    net.stop()

if __name__ == "__main__":
    main()

```

```

diz@diz:~/netprog-2026/lab05_custom_topo$ sudo python3 custom_topo.py
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s2) (5.00Mbit 50ms delay 10.00000% loss) (5.00Mbit 50ms delay 10.00000% loss) (s1, s2)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...(5.00Mbit 50ms delay 10.00000% loss) (5.00Mbit 50ms delay 10.00000% loss)
*** Ping: testing ping reachability
h1 -> h2 h3 X
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 X X
*** Results: 25% dropped (9/12 received)

```

Figure 3: Startup of the final script: TCLink reports (5.00Mbit 50ms delay 10.00000% loss) on both ends of the s1-s2 link

The startup log prints (5.00Mbit 50ms delay 10.00000% loss) twice — once for each interface of the s1-s2 link — and nothing for the host links, proving that the parameters apply only to the backbone. The initial `net.pingAll()` already lost 3 pings, all involving h4.

Task 3 — Bandwidth across the Backbone (Part F)

```
mininet> iperf h1 h4
*** Iperf: testing TCP bandwidth between h1 and h4
*** Results: ['395.3 kbits/sec', '454.5 kbits/sec']
```

Figure 4: iperf h1 h4 across the constrained backbone

```
mininet> iperfudp 10M h1 h4
*** Iperf: testing UDP bandwidth between h1 and h4
*** Results: ['10M', '4.9 Mbits/sec', '10.5 Mbits/sec']
```

Figure 5: iperfudp 10M h1 h4: h1 sends 10.5 Mbit/s, but h4 receives only 4.9 Mbit/s

Measurement	Lab 5.1 (no TCLink)	Lab 5.2 (backbone with TCLink)
TCP iperf h1 h4	30.0 Gbits/sec	395.3 kbits/sec / 454.5 kbits/sec
UDP iperfudp 10M h1 h4	—	sent 10.5 Mbits/sec, received 4.9 Mbits/sec
Configured cap	none	5 Mbit/s, 50 ms delay, 10% loss

The throughput dropped from the multi-Gbit/s figure of Lab 5.1 to under 0.5 Mbit/s, so the backbone is clearly constrained and no longer limited only by the CPU. The result is well *below* the 5 Mbit/s cap, however, because the same link also has 10% loss and 100 ms RTT. TCP treats every lost segment as congestion and halves its sending window, and with a 100 ms RTT it grows back slowly, so it never reaches the 5 Mbit/s ceiling. The Mathis TCP model predicts this: $\text{throughput} \approx (\text{MSS} / \text{RTT}) \times 1.22 / \sqrt{p} = (1460 \times 8 \text{ bit} / 0.1 \text{ s}) \times 1.22 / \sqrt{0.1} \approx \mathbf{450 \text{ kbit/s}}$, which matches the 454.5 kbit/s measured. The 5 Mbit/s cap is therefore an upper bound that loss prevents TCP from reaching.

To confirm the cap itself, I repeated the test with UDP, which has no congestion control and keeps sending at a fixed rate even when packets are lost. With `iperfudp 10M h1 h4`, h1 offered 10.5 Mbit/s but h4 received only **4.9 Mbit/s** — right at the configured `bw=5`. The output format is [requested rate, rate received by the server, rate sent by the client]. The excess traffic was dropped by the link's rate limiter, so the backbone is correctly capped near 5 Mbit/s.

Task 4 — Latency: Across the Backbone vs. Same Switch (Part F)

```

mininet> h1 ping -c 5 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=105 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=102 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=100 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=101 ms

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 100.274/101.678/104.844/1.670 ms
mininet>
mininet>
mininet> h1 ping -c 5 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.99 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.613 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.116 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.124 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.107 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4093ms
rtt min/avg/max/mdev = 0.107/0.590/1.993/0.727 ms
mininet>

```

Figure 6: h1 ping -c 5 h4 (across the backbone) and h1 ping -c 5 h2 (same switch)

Path	Links crossed	RTT min / avg / max (ms)	Average RTT
h1 → h4	h1-s1, s1-s2 , s2-h4	100.274 / 101.678 / 104.844	101.678 ms
h1 → h2	h1-s1, s1-h2	0.107 / 0.590 / 1.993	0.590 ms

The h1 → h4 ping crosses the backbone, which has `delay='50ms'`. The echo request is delayed 50 ms leaving s1 towards s2 and the echo reply is delayed another 50 ms leaving s2 towards s1, so the RTT is $2 \times 50 = 100$ ms plus the normal ≈ 0.1 – 1 ms of switching, giving the measured ≈ 101.7 ms. The h1 → h2 ping stays on s1 and never touches the backbone, so it keeps the fast sub-millisecond RTT of an unconstrained link. The difference (≈ 101.1 ms) is the ≈ 100 ms targeted. All 5 pings to h4 happened to arrive, which is possible with 10% loss (probability $\approx 0.81^5 \approx 35\%$).

Task 5 — Packet Loss on the Backbone (Part F)

```

mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 X
h3 -> h1 h2 h4
h4 -> h1 X h3
*** Results: 16% dropped (10/12 received)
mininet>
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 X
*** Results: 8% dropped (11/12 received)
mininet>
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 X
*** Results: 8% dropped (11/12 received)

```

Figure 7: Three consecutive pingall runs showing nonzero (but not 100%) drop rates

Run	Result	Failed pings
Initial net.pingAll()	25% dropped (9/12)	h1→h4, h4→h2, h4→h3
pingall #1	16% dropped (10/12)	h2→h4, h4→h2
pingall #2	8% dropped (11/12)	h4→h3
pingall #3	8% dropped (11/12)	h4→h3

Every run had a nonzero drop rate, but never 100%, and **every failed ping involved h4** — the only host behind the lossy s1–s2 link. Pings between h1, h2 and h3 never failed because they stay on s1. Of the 12 pings in a pingall, 6 cross the backbone, and each one must survive the link twice (request and reply): $P(\text{success}) = 0.9 \times 0.9 = 0.81$, so about 19% of them fail, or about 1.1 of 12 per run ($\approx 9.5\%$). Over the four runs I saw 7 failures out of 48 pings ($\approx 14.6\%$), which fits the random 10% per-packet loss, with some run-to-run variation.

Task 6 — Why Python Topologies Are More Powerful

The `--topo` generators can only create fixed, symmetric shapes with identical, unconstrained links. In Python I built **CustomTopo**, where s1 carries three hosts and s2 only one, and I gave just the s1–s2 backbone its own 5 Mbit/s, 50 ms, 10% loss characteristics while the host links stayed fast — neither the asymmetric shape nor per-link parameters can be expressed with single, linear or tree. Because the topology is code, it can also use loops, variables and functions, and it can be saved in Git and rerun exactly the same way each time.

3. Verification Checklist

#	Checkpoint	Evidence	✓
1	MinimalTopo (Part B) runs and net.pingAll() reports 0% dropped	Figure 1	✓
2	CustomTopo's net output (Part D) shows the correct asymmetric shape (3 hosts on s1, 1 host on s2)	Figure 2	✓
3	Mininet(topo=CustomTopo(), link=TCLink) includes link=TCLink (Part E)	Code, Figure 3	✓

#	Checkpoint	Evidence	✓
4	iperf h1 h4 reports a bandwidth close to the configured 5 Mbit/s cap, not an uncapped multi-Gbit/s figure — UDP: 4.9 Mbit/s received; TCP ≈0.4 Mbit/s due to 10% loss, see Task 3	Figures 4–5	✓
5	h1-h4 ping RTT is noticeably higher than h1-h2 ping RTT, consistent with the configured 50ms delay (101.678 ms vs. 0.590 ms)	Figure 6	✓
6	Repeated pingall runs show at least one nonzero drop rate consistent with loss=10	Figure 7	✓

4. Reflection Questions

Q1. What is the role of build() in a Topo subclass — when is it called, and by what?

`build()` is where the topology is *described*: it only declares hosts, switches and links with `addHost()`, `addSwitch()` and `addLink()`, and creates nothing real yet. I never call it myself — the `Topo` base-class constructor calls `build()` automatically when I write `CustomTopo()`, which happens as the argument to `Mininet(topo=CustomTopo())`. `Mininet` then reads that description and creates the real network namespaces, switches and veth links, which come up on `net.start()`.

Q2. Why must link=TCLink be passed to Mininet() for bw, delay and loss to take effect?

`addLink()` only stores the parameters as options; the `link` argument of `Mininet()` selects which class actually creates each link. The default `Link` class makes a plain veth pair and ignores `bw`, `delay` and `loss` with no warning. `TCLink` uses `TCIntf` interfaces that call Linux traffic control (`tc`) to add an HTB rate limiter for `bw` and a netem queue for `delay` and `loss` on each end of the link — which is why the startup log printed the parameters twice for the s1-s2 link.

Q3. A real scenario that single, linear and tree cannot represent

A company with a headquarters LAN and a small branch office: the HQ switch has many hosts and a server on fast 1 Gbit/s links, the branch switch has two or three hosts, and the two sites are joined by a 10 Mbit/s, 30 ms leased-line or VPN link, perhaps with a backup 4G link that has 2% loss. A similar case from my research is an SDN vehicular network, where a roadside unit connects vehicles over a lossy, higher-delay wireless hop but connects to the video (DASH) server over a fast wired backbone. These need different numbers of hosts per switch, redundant paths and different parameters on each link, which none of the built-in generators can produce.

5. Conclusion

Both custom topologies were built from Python and worked correctly (0% dropped). Adding `link=TCLink` and `bw=5`, `delay='50ms'`, `loss=10` to the s1-s2 link turned it into a WAN-like backbone: RTT across it rose to ≈101.7 ms versus 0.59 ms inside s1, every `pingall` showed random losses involving only h4, UDP throughput was capped at 4.9 Mbit/s, and TCP throughput fell from 30 Gbit/s to ≈0.4 Mbit/s — below the cap because TCP slows down when packets are lost on a long-RTT path.